

A Technical Introduction to Graph neural network

$$\mathbf{a}^{(l)} = \sigma(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

Section 1

Graph neural networks are one of the main building blocks of AlphaFold, an artificial intelligence program developed by Google's DeepMind for solving the protein folding problem in biology. AlphaFold achieved first place in several CASP competitions.

Section 2

Graph convolutional network The graph convolutional network (GCN) was first introduced by Thomas Kipf and Max Welling in 2017. A GCN layer defines a first-order approximation of a localized spectral filter on graphs. GCNs can be understood as a generalization of convolutional neural networks to graph-structured data. The formal expression of a GCN layer reads as follows:

Section 3

In the more general subject of "geometric deep learning", certain existing neural network architectures can be interpreted as GNNs operating on suitably defined graphs. A convolutional neural network layer, in the context of computer vision, can be considered a GNN applied to graphs whose nodes are pixels, and only adjacent pixels are connected by edges in the graph. A transformer layer, in natural language processing, can be considered a GNN applied to complete graphs whose nodes are words or tokens in a passage of natural language text. Relevant application domains for GNNs include natural language processing, social networks, citation networks, molecular biology, chemistry, physics and NP-hard combinatorial optimization problems. Open source libraries implementing GNNs include PyTorch Geometric (PyTorch), TensorFlow GNN (TensorFlow), Deep Graph Library (framework agnostic), jraph (Google JAX), and GraphNeuralNetworks.jl/GeometricFlux.jl (Julia, Flux).

Section 4

where $\mathbf{a}$ is a vector of learnable weights, $\cdot^T$ indicates transposition, $\mathbf{e}_{uv}$ are the edge features (if present), and LeakyReLU is a modified ReLU activation function. Attention coefficients are then normalized via softmax to make them easily comparable across different nodes. A GCN can be seen as a special case of a GAT where attention coefficients are not learnable, but fixed and equal

to the edge weights w_{uv} .

Section 5

Message passing layers are permutation-equivariant layers mapping a graph into an updated representation of the same graph. Formally, they can be expressed as message passing neural networks (MPNNs). Let $G = (V, E)$ be a graph, where V is the node set and E is the edge set. Let N_u be the neighbourhood of some node $u \in V$. Additionally, let $\mathbf{x}_u$ be the features of node $u \in V$, and $\mathbf{e}_{uv}$ be the features of edge $(u, v) \in E$. An MPNN layer can be expressed as follows:

Section 6

Permutation equivariant: a permutation equivariant layer maps a representation of a graph into an updated representation of the same graph. In the literature, permutation equivariant layers are implemented via pairwise message passing between graph nodes. Intuitively, in a message passing layer, nodes update their representations by aggregating the messages received from their immediate neighbours. As such, each message passing layer increases the receptive field of the GNN by one hop. **Local pooling:** a local pooling layer coarsens the graph via downsampling. Local pooling is used to increase the receptive field of a GNN, in a similar fashion to pooling layers in convolutional neural networks. Examples include k-nearest neighbours pooling, top-k pooling, and self-attention pooling. **Global pooling:** a global pooling layer, also known as readout layer, provides fixed-size representation of the whole graph. The global pooling layer must be permutation invariant, such that permutations in the ordering of graph nodes and edges do not alter the final output. Examples include element-wise sum, mean or maximum. It has been demonstrated that GNNs cannot be more expressive than the Weisfeiler–Leman Graph Isomorphism Test. In practice, this means that there exist different graph structures (e.g., molecules with the same atoms but different bonds) that cannot be distinguished by GNNs. More powerful GNNs operating on higher-dimension geometries such as simplicial complexes can be designed. As of 2022, whether or not future architectures will overcome the message passing primitive is an open research question.

Section 7

Scalability and distributed training Training GNNs on large graphs presents computational challenges that differ substantially from those in other neural network architectures. Two broad problem settings arise: training on a single very large graph (such as a social network with billions of nodes), and training on collections of large

individual graphs (such as molecular systems requiring higher-order interaction modeling). For the single large graph setting, sampling-based methods reduce memory requirements by operating on subgraphs rather than the full graph. Cluster-GCN partitions the graph into subgraphs using graph partitioning algorithms, training the model on each partition in sequence. GraphSAINT instead samples subgraphs stochastically during training, constructing unbiased estimators for the full graph loss. Distributed frameworks such as DistDGL extend training to multi-machine settings, managing the inter-machine communication required for neighborhood aggregation when a graph's nodes are distributed across machines. For tasks such as atomic simulations, a different bottleneck arises: GNN architectures that model higher-order interactions between triplets or quadruplets of atoms are memory-intensive at the level of individual graphs, making standard data parallelism — where each GPU processes a separate sample — insufficient when individual graphs exceed single-device memory. Graph Parallelism addresses this by distributing a single input graph across multiple GPUs, partitioning nodes and edges across devices rather than distributing samples across devices.. This approach enabled GNNs with hundreds of millions to billions of parameters to be trained on atomic systems that would otherwise exceed single-device memory, and has been applied to the development of large-scale universal interatomic potentials.

Section 8

$$\mathbf{h}_u^{(l+1)} = \text{GRU}(\mathbf{m}_u^{(l+1)}, \mathbf{h}_u^{(l)})$$

Section 9

Gated graph sequence neural network The gated graph sequence neural network (GGS-NN) was introduced by Yujia Li et al. in 2015. The GGS-NN extends the GNN formulation by Scarselli et al. to output sequences. The message passing framework is implemented as an update rule to a gated recurrent unit (GRU) cell. A GGS-NN can be expressed as follows: